

Interview Preparation Codes

ARRAY 1D

1. **Find the Maximum and Minimum Elements:**
 - **Question:** Write a function to find the maximum and minimum elements in an array.
 - **Logic:** Iterate through the array, updating the maximum and minimum as you go.
 - **Sample Input:** [5, 3, 9, 2, 8]
 - **Expected Output:** Maximum: 9, Minimum: 2
2. **Find the Second Largest Element:**
 - **Question:** Find the second largest element in an array.
 - **Logic:** Initialize two variables to store the largest and second-largest elements and iterate through the array.
 - **Sample Input:** [7, 3, 9, 2, 8]
 - **Expected Output:** Second Largest: 8
3. **Remove Duplicates from Unsorted Array:**
 - **Question:** Write a function to remove duplicates from an unsorted array.
 - **Logic:** Use a hash set to keep track of unique elements as you iterate through the array.
 - **Sample Input:** [3, 2, 3, 1, 2, 4]
 - **Expected Output:** [3, 2, 1, 4]
4. **Find Missing Number in Array:**
 - **Question:** Given an array containing n distinct numbers taken from 0, 1, 2, ..., n, find the one that is missing from the array.
 - **Logic:** Calculate the expected sum of the series and subtract the sum of the array from it.
 - **Sample Input:** [0, 1, 3]
 - **Expected Output:** Missing Number: 2
5. **Intersection of Two Arrays:**
 - **Question:** Find the intersection of two arrays.
 - **Logic:** Use a hash set to store one array's elements and check for common elements in the second array.
 - **Sample Input:** [1, 2, 2, 1] and [2, 2]
 - **Expected Output:** [2]
6. **Array Rotation:**
 - **Question:** Rotate an array to the right by a given number of steps.
 - **Logic:** Use array slicing or create a new array to rearrange elements according to the rotation steps.
 - **Sample Input:** [1, 2, 3, 4, 5] rotated by 2 steps
 - **Expected Output:** [4, 5, 1, 2, 3]
7. **Largest Subarray with Zero Sum:**
 - **Question:** Find the largest subarray with a sum of zero.

Interview Preparation Codes

- **Logic:** Use a hash map to keep track of the prefix sum and its corresponding index.
 - **Sample Input:** [4, 2, -3, 1, 6]
 - **Expected Output:** Subarray with Zero Sum: [2, -3, 1]
8. **Rearrange Positive and Negative Numbers:**
- **Question:** Given an array containing both positive and negative numbers, rearrange them in an alternating fashion.
 - **Logic:** Separate positive and negative numbers, then merge them alternately.
 - **Sample Input:** [-1, 2, -3, 4, 5, -6]
 - **Expected Output:** [-1, 2, -3, 4, -6, 5]
9. **Find the Majority Element:**
- **Question:** Find the majority element (element that appears more than $n/2$ times) in an array.
 - **Logic:** Use Moore's Voting Algorithm to find the majority element.
 - **Sample Input:** [3, 3, 4, 2, 4, 4, 2, 4, 4]
 - **Expected Output:** Majority Element: 4
10. **Product of Array Except Self:**
- **Question:** Given an array, return an array where each element is the product of all the elements in the array except itself.
 - **Logic:** Use two passes, one from left to right, and one from right to left, to calculate products.
 - **Sample Input:** [1, 2, 3, 4]
 - **Expected Output:** [24, 12, 8, 6]
11. **Move Zeros to the End:**
- **Question:** Given an array, move all the zeros to the end without changing the order of non-zero elements.
 - **Logic:** Iterate through the array, moving non-zero elements to the front and filling the remaining space with zeros.
 - **Sample Input:** [0, 1, 0, 3, 12]
 - **Expected Output:** [1, 3, 12, 0, 0]
12. **Dutch National Flag Problem:**
- **Question:** Given an array containing only 0s, 1s, and 2s, sort the array in a single pass.
 - **Logic:** Use three pointers to maintain three regions in the array.
 - **Sample Input:** [2, 0, 1, 2, 1, 0]
 - **Expected Output:** [0, 0, 1, 1, 2, 2]
13. **Find the Kth Largest Element:**
- **Question:** Find the Kth largest element in an array.
 - **Logic:** Use techniques like quickselect or a max-heap to find the Kth largest element efficiently.
 - **Sample Input:** [3, 2, 1, 5, 6, 4], K = 2
 - **Expected Output:** 5
14. **Sort an Array of 0s, 1s, and 2s (Dutch National Flag Variant):**

Interview Preparation Codes

- **Question:** Given an array with only 0s, 1s, and 2s, sort the array in a single pass without using extra space.
 - **Logic:** Use a modified Dutch National Flag algorithm.
 - **Sample Input:** [1, 0, 2, 1, 0, 2]
 - **Expected Output:** [0, 0, 1, 1, 2, 2]
15. **Find the Intersection of Three Arrays:**
- **Question:** Find the common elements in three arrays.
 - **Logic:** Use three sets to keep track of common elements between the arrays.
 - **Sample Input:** [1, 2, 3], [2, 3, 4], [3, 4, 5]
 - **Expected Output:** [3]
16. **Spiral Matrix Traversal:**
- **Question:** Given a matrix, traverse it in a spiral order (clockwise).
 - **Logic:** Use four pointers to define the boundaries of the matrix and traverse in a spiral order.
 - **Sample Input:**

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
 - **Expected Output:** [1, 2, 3, 6, 9, 8, 7, 4, 5]
17. **Subarray Sum Equals K:**
- **Question:** Given an array of integers and an integer K, find the total number of continuous subarrays whose sum equals K.
 - **Logic:** Use a hashmap to keep track of prefix sums and count subarrays with a specific difference.
 - **Sample Input:** [1, 1, 1], K = 2
 - **Expected Output:** 2
18. **Maximum Consecutive Ones:**
- **Question:** Find the maximum number of consecutive 1s in a binary array.
 - **Logic:** Iterate through the array, keeping track of the current consecutive 1s and the maximum seen so far.
 - **Sample Input:** [1, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1]
 - **Expected Output:** 4
19. **Merge Intervals:**
- **Question:** Given a collection of intervals, merge any overlapping intervals.
 - **Logic:** Sort the intervals by their start points and merge overlapping intervals.
 - **Sample Input:** [[1, 3], [2, 6], [8, 10], [15, 18]]
 - **Expected Output:** [[1, 6], [8, 10], [15, 18]]
20. **Stock Buy and Sell:**
- **Question:** Given an array representing stock prices, find the maximum profit by buying and selling the stock.

Interview Preparation Codes

- **Logic:** Track the minimum price and maximum profit as you iterate through the array.
- **Sample Input:** [7, 1, 5, 3, 6, 4]
- **Expected Output:** 5

Interview Preparation Codes

Vector/ArrayList/List Handling:

1. **Find the Maximum Element in a List:**
 - **Question:** Write a function to find the largest element in a list.
 - **Logic:** Iterate through the list and keep track of the maximum element.
 - **Sample Input:** [3, 8, 1, 10, 5]
 - **Expected Output:** 10
2. **Remove Duplicates from a List:**
 - **Question:** Write a function to remove duplicate elements from a list.
 - **Logic:** Use a loop or convert the list to a set and back to a list.
 - **Sample Input:** [1, 2, 2, 3, 4, 4, 5]
 - **Expected Output:** [1, 2, 3, 4, 5]
3. **Reverse a List:**
 - **Question:** Write a function to reverse the order of elements in a list.
 - **Logic:** Use list slicing or a loop to reverse the elements.
 - **Sample Input:** [1, 2, 3, 4, 5]
 - **Expected Output:** [5, 4, 3, 2, 1]
4. **Check for Palindromic List:**
 - **Question:** Write a function to check if a list is a palindrome.
 - **Logic:** Compare the list with its reverse.
 - **Sample Input:** [1, 2, 3, 2, 1]
 - **Expected Output:** Palindrome
5. **Find Common Elements in Two Lists:**
 - **Question:** Write a function to find common elements between two lists.
 - **Logic:** Use list comprehensions or a loop to compare elements.
 - **Sample Input:** [1, 2, 3, 4] and [3, 4, 5, 6]
 - **Expected Output:** [3, 4]
6. **Merge Two Sorted Lists:**
 - **Question:** Write a function to merge two sorted lists into a single sorted list.
 - **Logic:** Use two pointers to iterate through the lists and merge them.
 - **Sample Input:** [1, 3, 5] and [2, 4, 6]
 - **Expected Output:** [1, 2, 3, 4, 5, 6]
7. **Calculate the Sum of List Elements:**
 - **Question:** Write a function to calculate the sum of all elements in a list.
 - **Logic:** Use a loop to iterate through the list and accumulate the sum.
 - **Sample Input:** [1, 2, 3, 4, 5]
 - **Expected Output:** 15
8. **Count Occurrences of an Element in a List:**
 - **Question:** Write a function to count how many times a specific element appears in a list.
 - **Logic:** Use a loop to count occurrences.
 - **Sample Input:** [1, 2, 2, 3, 4, 2] and 2
 - **Expected Output:** 3

Interview Preparation Codes

9. Find the Second Largest Element in a List:

- **Question:** Write a function to find the second largest element in a list.
- **Logic:** Iterate through the list and keep track of the largest and second largest elements.
- **Sample Input:** [3, 8, 1, 10, 5]
- **Expected Output:** 8

10. Generate a List of Prime Numbers:

- **Question:** Write a function to generate a list of prime numbers within a given range.
- **Logic:** Use loops and checks to find prime numbers.
- **Sample Input:** Range: 1 to 20
- **Expected Output:** List of Prime Numbers: [2, 3, 5, 7, 11, 13, 17, 19]

11. Add an Element at the Beginning of a List:

- **Question:** Write a function to add an element at the beginning of a list.
- **Logic:** Use list slicing or the `insert()` method.
- **Sample Input:** List: [2, 3, 4] and Element: 1
- **Expected Output:** List: [1, 2, 3, 4]

12. Remove an Element from a List by Value:

- **Question:** Write a function to remove a specific element from a list by its value.
- **Logic:** Use list methods like `remove()` or list comprehensions.
- **Sample Input:** List: [1, 2, 3, 4, 5] and Element: 3
- **Expected Output:** List: [1, 2, 4, 5]

13. Find the Third Largest Element in a List:

- **Question:** Write a function to find the third largest element in a list.
- **Logic:** Iterate through the list while keeping track of the largest, second largest, and third largest elements.
- **Sample Input:** [5, 9, 2, 7, 3, 8, 6]
- **Expected Output:** 7

14. Count Negative and Positive Numbers in a List:

- **Question:** Write a function to count the number of negative and positive numbers in a list.
- **Logic:** Use a loop to iterate through the list and keep two counters.
- **Sample Input:** [3, -2, 7, -1, 0, 5, -4]
- **Expected Output:** Positive: 4, Negative: 3

15. Find the Kth Smallest Element in a List:

- **Question:** Write a function to find the Kth smallest element in a list.
- **Logic:** Use sorting or a selection algorithm to find the Kth smallest element.
- **Sample Input:** List: [5, 2, 8, 3, 1, 9, 4], K: 3
- **Expected Output:** 3

16. Remove All Occurrences of an Element in a List:

- **Question:** Write a function to remove all occurrences of a specific element from a list.

Interview Preparation Codes

- **Logic:** Use list comprehensions or a loop to filter out the element.
 - **Sample Input:** List: [1, 2, 2, 3, 4, 2] and Element: 2
 - **Expected Output:** List: [1, 3, 4]
17. **Calculate the Product of List Elements:**
- **Question:** Write a function to calculate the product of all elements in a list.
 - **Logic:** Use a loop to iterate through the list and accumulate the product.
 - **Sample Input:** [2, 3, 4, 5]
 - **Expected Output:** 120
18. **Find the First Non-Repeating Element in a List:**
- **Question:** Write a function to find the first non-repeating element in a list.
 - **Logic:** Use a loop and a dictionary to count occurrences.
 - **Sample Input:** [1, 2, 3, 2, 1, 4, 5]
 - **Expected Output:** 3
19. **Split a List into Chunks of a Given Size:**
- **Question:** Write a function to split a list into smaller lists of a specified size.
 - **Logic:** Use list comprehensions or a loop to create chunks.
 - **Sample Input:** List: [1, 2, 3, 4, 5, 6, 7, 8], Chunk Size: 3
 - **Expected Output:** [[1, 2, 3], [4, 5, 6], [7, 8]]
20. **Find the Union of Two Lists:**
- **Question:** Write a function to find the union of two lists.
 - **Logic:** Use list comprehensions or a loop to combine the lists.
 - **Sample Input:** List 1: [1, 2, 3], List 2: [3, 4, 5]
 - **Expected Output:** [1, 2, 3, 4, 5]
21. **Check if List is Sorted in Non-Decreasing Order:**
- **Question:** Write a function to check if a list is sorted in non-decreasing order.
 - **Logic:** Use a loop to compare adjacent elements.
 - **Sample Input:** [1, 2, 3, 3, 4, 5]
 - **Expected Output:** Sorted
22. **Remove Leading Zeros from a List of Integers:**
- **Question:** Write a function to remove leading zeros from a list of integers.
 - **Logic:** Use list slicing or a loop to remove zeros until a non-zero element is encountered.
 - **Sample Input:** [0, 0, 1, 2, 0, 3, 0, 0, 4]
 - **Expected Output:** [1, 2, 0, 3, 0, 0, 4]
23. **Find the Largest Subarray with Contiguous Elements:**
- **Question:** Write a function to find the largest subarray with contiguous elements in a list.
 - **Logic:** Use a loop and keep track of the longest contiguous subarray.

Interview Preparation Codes

- **Sample Input:** [1, 2, 3, 4, 6, 7, 8, 9, 10]
 - **Expected Output:** [1, 2, 3, 4]
24. **Find the First Missing Positive Integer:**
- **Question:** Write a function to find the first missing positive integer in a list of unsorted integers.
 - **Logic:** Use a loop and reposition elements to place each positive integer at its respective index.
 - **Sample Input:** [3, 4, -1, 1]
 - **Expected Output:** 2
25. **Find the Smallest Missing Positive Integer:**
- **Question:** Write a function to find the smallest missing positive integer in a list of unsorted integers.
 - **Logic:** Use a loop and reposition elements to place each positive integer at its respective index.
 - **Sample Input:** [7, 8, 9, 11, 12]
 - **Expected Output:** 1
26. **Find the Longest Subarray with Equal 0s and 1s:**
- **Question:** Write a function to find the longest subarray with an equal number of 0s and 1s in a list of binary integers.
 - **Logic:** Use a loop and keep track of the longest subarray with a balance of 0s and 1s.
 - **Sample Input:** [0, 1, 0, 0, 1, 1, 0]
 - **Expected Output:** [0, 1, 0, 0, 1, 1]
27. **Merge Overlapping Intervals:**
- **Question:** Write a function to merge overlapping intervals in a list.
 - **Logic:** Use sorting and a loop to merge overlapping intervals.
 - **Sample Input:** [[1, 3], [2, 6], [8, 10], [15, 18]]
 - **Expected Output:** [[1, 6], [8, 10], [15, 18]]
28. **Rearrange Elements in Zigzag Order:**
- **Question:** Write a function to rearrange elements in a list in zigzag order ($a < b > c < d > e$).
 - **Logic:** Use a loop to swap elements according to the zigzag pattern.
 - **Sample Input:** [4, 3, 7, 8, 6, 2, 1]
 - **Expected Output:** [3, 7, 4, 8, 2, 6, 1]
29. **Find the Maximum Subarray Sum:**
- **Question:** Write a function to find the maximum sum of a subarray in a list.
 - **Logic:** Use Kadane's algorithm or a loop to calculate the maximum subarray sum.
 - **Sample Input:** [-2, 1, -3, 4, -1, 2, 1, -5, 4]
 - **Expected Output:** 6
30. **Find All Duplicates in a List:**
- **Question:** Write a function to find all the elements that appear more than once in a list.

Interview Preparation Codes

- **Logic:** Use a loop and a dictionary to count occurrences.
- **Sample Input:** [4, 3, 2, 7, 8, 2, 1, 5, 5]
- **Expected Output:** [2, 5]

Interview Preparation Codes

Dictionary Handling:

1. **Access Dictionary Values:**
 - **Question:** Write a function to access values from a dictionary using keys.
 - **Logic:** Use square brackets or the `get()` method.
 - **Sample Input:** Dictionary: {"name": "Alice", "age": 30}, Key: "name"
 - **Expected Output:** "Alice"
2. **Add Key-Value Pairs to a Dictionary:**
 - **Question:** Write a function to add key-value pairs to a dictionary.
 - **Logic:** Use square brackets or the `update()` method.
 - **Sample Input:** Dictionary: {"name": "Alice"}, Key: "age", Value: 30
 - **Expected Output:** Dictionary: {"name": "Alice", "age": 30}
3. **Remove Key-Value Pairs from a Dictionary:**
 - **Question:** Write a function to remove key-value pairs from a dictionary.
 - **Logic:** Use the `pop()` method or the `del` statement.
 - **Sample Input:** Dictionary: {"name": "Alice", "age": 30}, Key: "age"
 - **Expected Output:** Dictionary: {"name": "Alice"}
4. **Check if a Key Exists in a Dictionary:**
 - **Question:** Write a function to check if a key exists in a dictionary.
 - **Logic:** Use the `in` operator or the `get()` method.
 - **Sample Input:** Dictionary: {"name": "Alice", "age": 30}, Key: "age"
 - **Expected Output:** Key Exists
5. **Iterate Over Dictionary Keys and Values:**
 - **Question:** Write a function to iterate over the keys and values of a dictionary.
 - **Logic:** Use loops to access keys and values.
 - **Sample Input:** Dictionary: {"name": "Alice", "age": 30}
 - **Expected Output:** Iteration over keys and values.
6. **Merge Two Dictionaries:**
 - **Question:** Write a function to merge two dictionaries into a single dictionary.
 - **Logic:** Use the `update()` method or dictionary unpacking.
 - **Sample Input:** Dict1: {"name": "Alice"}, Dict2: {"age": 30}
 - **Expected Output:** Merged Dictionary: {"name": "Alice", "age": 30}
7. **Count Frequency of Elements in a List using a Dictionary:**
 - **Question:** Write a function to count the frequency of elements in a list using a dictionary.
 - **Logic:** Iterate through the list and use a dictionary to store counts.
 - **Sample Input:** [1, 2, 2, 3, 4, 3, 5]
 - **Expected Output:** {"1": 1, "2": 2, "3": 2, "4": 1, "5": 1}
8. **Check if a Dictionary is Empty:**
 - **Question:** Write a function to check if a dictionary is empty.
 - **Logic:** Use the `not` operator or check the length of the dictionary.

Interview Preparation Codes

- **Sample Input:** Dictionary: {}
 - **Expected Output:** Empty
9. **Find the Key with the Maximum Value in a Dictionary:**
- **Question:** Write a function to find the key with the maximum value in a dictionary.
 - **Logic:** Iterate through the dictionary and keep track of the key with the maximum value.
 - **Sample Input:** {"A": 50, "B": 30, "C": 70}
 - **Expected Output:** "C"
10. **Reverse a Dictionary:**
- **Question:** Write a function to reverse the key-value pairs of a dictionary.
 - **Logic:** Create a new dictionary with reversed key-value pairs.
 - **Sample Input:** {"A": 1, "B": 2, "C": 3}
 - **Expected Output:** {"1": "A", "2": "B", "3": "C"}
11. **Find Common Key-Value Pairs in Two Dictionaries:**
- **Question:** Write a function to find common key-value pairs between two dictionaries.
 - **Logic:** Iterate through one dictionary and check if the keys exist in the other dictionary with matching values.
 - **Sample Input:** Dict1: {"A": 1, "B": 2, "C": 3}, Dict2: {"B": 2, "C": 4, "D": 5}
 - **Expected Output:** {"B": 2}
12. **Find the Key with the Minimum Value in a Dictionary:**
- **Question:** Write a function to find the key with the minimum value in a dictionary.
 - **Logic:** Iterate through the dictionary and keep track of the key with the minimum value.
 - **Sample Input:** {"X": 20, "Y": 10, "Z": 30}
 - **Expected Output:** "Y"
13. **Remove Duplicates from a List of Dictionaries:**
- **Question:** Write a function to remove duplicates from a list of dictionaries based on a specific key.
 - **Logic:** Use a loop to iterate through the list and a set to track seen values of the key.
 - **Sample Input:** [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}, {"name": "Alice", "age": 30}]
 - **Expected Output:** Uniqued List: [{"name": "Alice", "age": 30}, {"name": "Bob", "age": 25}]
14. **Find the Most Frequent Element in a List:**
- **Question:** Write a function to find the most frequent element in a list.
 - **Logic:** Iterate through the list and use a dictionary to count occurrences.
 - **Sample Input:** [1, 2, 2, 3, 4, 2, 5, 2]
 - **Expected Output:** Most Frequent Element: 2

Interview Preparation Codes

15. Check if Two Dictionaries are Equal:

- **Question:** Write a function to check if two dictionaries are equal (have the same key-value pairs).
- **Logic:** Iterate through both dictionaries and compare key-value pairs.
- **Sample Input:** Dict1: {"A": 1, "B": 2, "C": 3}, Dict2: {"C": 3, "B": 2, "A": 1}
- **Expected Output:** Equal

16. Find the Median Value in a Dictionary:

- **Question:** Write a function to find the median value in a dictionary.
- **Logic:** Extract values from the dictionary, sort them, and find the median.
- **Sample Input:** {"A": 10, "B": 20, "C": 30, "D": 40, "E": 50}
- **Expected Output:** Median Value: 30

17. Calculate the Intersection of Two Dictionaries:

- **Question:** Write a function to calculate the intersection of two dictionaries (common key-value pairs).
- **Logic:** Iterate through one dictionary and check if the keys exist in the other dictionary with matching values.
- **Sample Input:** Dict1: {"A": 1, "B": 2, "C": 3}, Dict2: {"B": 2, "C": 4, "D": 5}
- **Expected Output:** {"B": 2}

18. Check if a Dictionary is a Subset of Another:

- **Question:** Write a function to check if one dictionary is a subset of another.
- **Logic:** Iterate through the first dictionary and check if its key-value pairs exist in the second dictionary.
- **Sample Input:** Subset Dict: {"A": 1, "B": 2}, Superset Dict: {"A": 1, "B": 2, "C": 3}
- **Expected Output:** Subset

19. Remove Specific Keys from a Dictionary:

- **Question:** Write a function to remove specific keys from a dictionary.
- **Logic:** Iterate through the dictionary and remove keys that match the specified keys.
- **Sample Input:** Dictionary: {"A": 1, "B": 2, "C": 3}, Keys to Remove: ["A", "C"]
- **Expected Output:** Dictionary: {"B": 2}

20. Find the Sum of All Values in a Dictionary:

- **Question:** Write a function to calculate the sum of all values in a dictionary.
- **Logic:** Iterate through the dictionary and accumulate the sum of values.
- **Sample Input:** {"A": 10, "B": 20, "C": 30, "D": 40}
- **Expected Output:** Sum of Values: 100

21. Find the Key with the Longest Value in a Dictionary:

- **Question:** Write a function to find the key with the longest value (string) in a dictionary.

Interview Preparation Codes

- **Logic:** Iterate through the dictionary and compare the lengths of string values.
 - **Sample Input:** {"A": "apple", "B": "banana", "C": "cherry", "D": "date"}
 - **Expected Output:** "C"
22. **Sort Dictionary by Key or Value:**
- **Question:** Write a function to sort a dictionary by keys or values in ascending or descending order.
 - **Logic:** Use the `sorted()` function with a custom key or use list comprehension.
 - **Sample Input:** {"C": 3, "B": 2, "A": 1}
 - **Expected Output (Ascending by Key):** {"A": 1, "B": 2, "C": 3}
 - **Expected Output (Descending by Value):** {"C": 3, "B": 2, "A": 1}
23. **Remove All Elements from a Dictionary:**
- **Question:** Write a function to remove all elements from a dictionary.
 - **Logic:** Use the `clear()` method or assign an empty dictionary.
 - **Sample Input:** {"A": 1, "B": 2, "C": 3}
 - **Expected Output:** Empty Dictionary: {}
24. **Count the Number of Non-Empty Values in a Dictionary:**
- **Question:** Write a function to count the number of non-empty (non-null) values in a dictionary.
 - **Logic:** Iterate through the dictionary and count values that are not empty.
 - **Sample Input:** {"A": 1, "B": "", "C": 3, "D": None, "E": "Five"}
 - **Expected Output:** Number of Non-Empty Values: 3
25. **Find the Difference Between Two Dictionaries:**
- **Question:** Write a function to find the difference between two dictionaries (key-value pairs that exist in one but not in the other).
 - **Logic:** Iterate through one dictionary and check if the keys exist in the other dictionary.
 - **Sample Input:** Dict1: {"A": 1, "B": 2, "C": 3}, Dict2: {"B": 2, "C": 4, "D": 5}
 - **Expected Output:** {"A": 1}
26. **Find the Largest Value in a Dictionary:**
- **Question:** Write a function to find the largest value in a dictionary.
 - **Logic:** Iterate through the dictionary and keep track of the largest value.
 - **Sample Input:** {"A": 10, "B": 20, "C": 30, "D": 40, "E": 50}
 - **Expected Output:** Largest Value: 50
27. **Count the Number of Key-Value Pairs in a Dictionary:**
- **Question:** Write a function to count the number of key-value pairs in a dictionary.
 - **Logic:** Use the `len()` function to count the number of items in the dictionary.
 - **Sample Input:** {"A": 1, "B": 2, "C": 3}
 - **Expected Output:** Number of Key-Value Pairs: 3

Interview Preparation Codes

28. Sort Dictionary by Key Length:

- **Question:** Write a function to sort a dictionary by the length of its keys.
- **Logic:** Use the `sorted()` function with a custom key function.
- **Sample Input:** {"One": 1, "Three": 3, "Two": 2}
- **Expected Output:** {"Two": 2, "One": 1, "Three": 3}

29. Find the Smallest Value in a Dictionary:

- **Question:** Write a function to find the smallest value in a dictionary.
- **Logic:** Iterate through the dictionary and keep track of the smallest value.
- **Sample Input:** {"X": 20, "Y": 10, "Z": 30}
- **Expected Output:** Smallest Value: 10

30. Check if a Dictionary is a Superset of Another:

- **Question:** Write a function to check if one dictionary is a superset of another.
- **Logic:** Iterate through the first dictionary and check if its key-value pairs exist in the second dictionary.
- **Sample Input:** Superset Dict: {"A": 1, "B": 2, "C": 3}, Subset Dict: {"A": 1, "B": 2}
- **Expected Output:** Superset

Interview Preparation Codes

String

1. **Reverse a String:**
 - **Question:** Write a program to reverse a given string.
 - **Logic:** Use a loop to reverse the characters in the string.
 - **Sample Input:** "hello"
 - **Expected Output:** "olleh"
2. **Check for Palindrome:**
 - **Question:** Write a program to check if a given string is a palindrome.
 - **Logic:** Use loops to compare characters from the start and end of the string.
 - **Sample Input:** "racecar"
 - **Expected Output:** Palindrome
3. **Count Vowels and Consonants:**
 - **Question:** Write a program to count the number of vowels and consonants in a given string.
 - **Logic:** Use loops and conditional statements to categorize characters.
 - **Sample Input:** "hello"
 - **Expected Output:** Vowels: 2, Consonants: 3
4. **Check for Anagrams:**
 - **Question:** Write a program to check if two strings are anagrams of each other.
 - **Logic:** Check if the character counts in both strings are the same.
 - **Sample Input:** "listen" and "silent"
 - **Expected Output:** Anagrams
5. **Check for Pangram:**
 - **Question:** Write a program to check if a given string is a pangram (contains all letters of the alphabet).
 - **Logic:** Use loops to check if all letters are present.
 - **Sample Input:** "The quick brown fox jumps over the lazy dog."
 - **Expected Output:** Pangram
6. **Remove Duplicates from a String:**
 - **Question:** Write a program to remove duplicate characters from a string.
 - **Logic:** Use a hash set or loops to track unique characters.
 - **Sample Input:** "programming"
 - **Expected Output:** "progamin"
7. **Count Substrings in a String:**
 - **Question:** Write a program to count the number of occurrences of a substring in a given string.
 - **Logic:** Use loops to search for the substring and count occurrences.
 - **Sample Input:** "abababab" and "ab"
 - **Expected Output:** 4

Interview Preparation Codes

8. Check for Rotation:

- **Question:** Write a program to check if one string is a rotation of another string.
- **Logic:** Concatenate one string with itself and check if the other string is a substring.
- **Sample Input:** "waterbottle" and "erbottlewat"
- **Expected Output:** Rotation

9. Remove White Spaces:

- **Question:** Write a program to remove white spaces from a string.
- **Logic:** Use loops to remove spaces or use a regular expression.
- **Sample Input:** "hello world"
- **Expected Output:** "helloworld"

10. Count Words in a String:

- **Question:** Write a program to count the number of words in a string.
- **Logic:** Use loops to count spaces and words.
- **Sample Input:** "This is a sentence."
- **Expected Output:** 4

11. Reverse Words in a String:

- **Question:** Write a program to reverse the order of words in a string.
- **Logic:** Split the string into words, reverse the order, and join them back together.
- **Sample Input:** "Hello world"
- **Expected Output:** "world Hello"

12. Title Case a Sentence:

- **Question:** Write a program to convert the first letter of each word to uppercase in a sentence.
- **Logic:** Split the sentence into words and capitalize the first letter of each word.
- **Sample Input:** "this is a test"
- **Expected Output:** "This Is A Test"

13. Check for Subsequence:

- **Question:** Write a program to check if one string is a subsequence of another.
- **Logic:** Use loops to check if the characters of the subsequence are in the same order.
- **Sample Input:** "abc" and "ahbgdc"
- **Expected Output:** Subsequence

14. Find Longest Common Prefix:

- **Question:** Write a program to find the longest common prefix among an array of strings.
- **Logic:** Compare characters of all strings from left to right.
- **Sample Input:** ["flower", "flour", "flight"]
- **Expected Output:** "fl"

15. Compress a String:

Interview Preparation Codes

- **Question:** Write a program to compress a string by replacing consecutive characters with their counts.
 - **Logic:** Use loops to count consecutive characters and create the compressed string.
 - **Sample Input:** "aaabbbcccc"
 - **Expected Output:** "a3b3c4"
16. **Reverse Each Word in a String:**
- **Question:** Write a program to reverse each word in a string.
 - **Logic:** Split the string into words, reverse each word, and join them back together.
 - **Sample Input:** "Hello world"
 - **Expected Output:** "olleH dlrow"
17. **Check for Valid Parentheses:**
- **Question:** Write a program to check if a string containing parentheses is valid.
 - **Logic:** Use a stack to keep track of open and close parentheses.
 - **Sample Input:** "{[()]}"
 - **Expected Output:** Valid
18. **Find the First Non-Repeating Character:**
- **Question:** Write a program to find the first non-repeating character in a string.
 - **Logic:** Use loops to count characters and find the first non-repeating character.
 - **Sample Input:** "leetcode"
 - **Expected Output:** "l"
19. **Check for a Valid Palindromic String:**
- **Question:** Write a program to check if a given string is a valid palindromic string after ignoring non-alphanumeric characters and considering case.
 - **Logic:** Use loops to compare characters while ignoring non-alphanumeric characters.
 - **Sample Input:** "A man, a plan, a canal: Panama"
 - **Expected Output:** Valid Palindrome
20. **Count and Say Sequence:**
- **Question:** Write a program to generate the nth term of the "Count and Say" sequence.
 - **Logic:** Use loops to generate the sequence based on the previous term.
 - **Sample Input:** 4
 - **Expected Output:** "1211"

Interview Preparation Codes

Function Handling:

1. **Add Two Numbers:**
 - **Question:** Write a function that takes two numbers as input and returns their sum.
 - **Logic:** Define a function that takes two parameters and returns their sum.
 - **Sample Input:** 4 and 6
 - **Expected Output:** 10
2. **Calculate Factorial:**
 - **Question:** Write a function to calculate the factorial of a given number.
 - **Logic:** Define a recursive function to calculate the factorial of a number.
 - **Sample Input:** 5
 - **Expected Output:** 120
3. **Check for Prime Number:**
 - **Question:** Write a function to check if a given number is prime.
 - **Logic:** Define a function that checks if the number is divisible by any numbers from 2 to its square root.
 - **Sample Input:** 11
 - **Expected Output:** Prime
4. **Reverse a String:**
 - **Question:** Write a function to reverse a given string.
 - **Logic:** Define a function to reverse the characters in the string.
 - **Sample Input:** "hello"
 - **Expected Output:** "olleh"
5. **Find the GCD (Greatest Common Divisor):**
 - **Question:** Write a function to find the GCD of two numbers using the Euclidean algorithm.
 - **Logic:** Define a function that uses a loop to repeatedly find the remainder until it becomes zero.
 - **Sample Input:** 12 and 18
 - **Expected Output:** 6
6. **Check for Armstrong Number:**
 - **Question:** Write a function to check if a number is an Armstrong number.
 - **Logic:** Define a function that checks if the number is equal to the sum of its own digits raised to the power of the number of digits.
 - **Sample Input:** 153
 - **Expected Output:** Armstrong Number
7. **Calculate Exponentiation:**
 - **Question:** Write a function to calculate the result of raising a number to a given exponent.

Interview Preparation Codes

- **Logic:** Define a function that uses a loop to multiply the number by itself for the specified number of times.
 - **Sample Input:** Number = 2, Exponent = 5
 - **Expected Output:** 32
8. **Generate Fibonacci Sequence:**
- **Question:** Write a function to generate the first N terms of the Fibonacci sequence.
 - **Logic:** Define a function that generates Fibonacci numbers iteratively.
 - **Sample Input:** 6
 - **Expected Output:** Fibonacci Sequence: [0, 1, 1, 2, 3, 5]
9. **Check for Prime Numbers in a Range:**
- **Question:** Write a function to find and return all prime numbers within a given range.
 - **Logic:** Define a function that iterates through the range and checks for prime numbers.
 - **Sample Input:** Range: 1 to 20
 - **Expected Output:** List of Prime Numbers: [2, 3, 5, 7, 11, 13, 17, 19]
10. **Calculate Compound Interest:**
- **Question:** Write a function to calculate the compound interest for a principal amount, rate, and time.
 - **Logic:** Define a function that uses a loop to calculate compound interest based on the formula.
 - **Sample Input:** Principal amount = 1000, Rate = 5%, Time = 3 years
 - **Expected Output:** 157.63
11. **Check for Perfect Number:**
- **Question:** Write a function to check if a number is a perfect number.
 - **Logic:** Define a function that checks if the number is equal to the sum of its proper divisors.
 - **Sample Input:** 28
 - **Expected Output:** Perfect Number
12. **Find the Nth Prime Number:**
- **Question:** Write a function that finds the Nth prime number.
 - **Logic:** Define a function that uses a loop to check for prime numbers and keeps track of how many primes you've found.
 - **Sample Input:** 5
 - **Expected Output:** 11 (the 5th prime number)
13. **Check for a Valid Palindromic String:**
- **Question:** Write a function to check if a given string is a valid palindromic string after ignoring non-alphanumeric characters and considering case.
 - **Logic:** Define a function that uses loops to compare characters while ignoring non-alphanumeric characters.
 - **Sample Input:** "A man, a plan, a canal: Panama"
 - **Expected Output:** Valid Palindrome

Interview Preparation Codes

14. Find the First Non-Repeating Character:

- **Question:** Write a function to find the first non-repeating character in a string.
- **Logic:** Define a function that uses loops to count characters and find the first non-repeating character.
- **Sample Input:** "leetcode"
- **Expected Output:** "l"

15. Generate Pascal's Triangle:

- **Question:** Write a function to generate the first N rows of Pascal's Triangle.
- **Logic:** Define a function that generates and returns the rows of Pascal's Triangle using nested loops.
- **Sample Input:** 4 (to generate the first 4 rows)
- **Expected Output:** List of Lists representing Pascal's Triangle.

16. Calculate the Sum of Digits:

- **Question:** Write a function to calculate the sum of the digits of a given number.
- **Logic:** Define a function that uses loops to extract and sum the digits of the number.
- **Sample Input:** 12345
- **Expected Output:** 15

17. Calculate the Mean (Average) of Numbers in a List:

- **Question:** Write a function to calculate the mean (average) of a list of numbers.
- **Logic:** Define a function that sums the numbers and divides by the count.
- **Sample Input:** [2, 4, 6, 8, 10]
- **Expected Output:** 6.0

18. Find the Longest Word in a Sentence:

- **Question:** Write a function to find the longest word in a sentence.
- **Logic:** Define a function that splits the sentence into words and finds the longest one.
- **Sample Input:** "The quick brown fox"
- **Expected Output:** "quick"

19. Check for Pangram:

- **Question:** Write a function to check if a given string is a pangram (contains all letters of the alphabet).
- **Logic:** Define a function that uses loops to check if all letters are present.
- **Sample Input:** "The quick brown fox jumps over the lazy dog."
- **Expected Output:** Pangram

20. Count the Number of Words in a Sentence:

- **Question:** Write a function to count the number of words in a sentence.

Interview Preparation Codes

- **Logic:** Define a function that counts the spaces between words.
 - **Sample Input:** "This is a sentence."
 - **Expected Output:** 4
21. **Generate a Random Password:**
- **Question:** Write a function to generate a random password of a specified length.
 - **Logic:** Define a function that combines random characters from a character set.
 - **Sample Input:** 8 (for an 8-character password)
 - **Expected Output:** A random password, e.g., "A7&b2#z9"
22. **Calculate Simple Interest:**
- **Question:** Write a function to calculate the simple interest for a principal amount, rate, and time.
 - **Logic:** Define a function that uses the simple interest formula.
 - **Sample Input:** Principal amount = 1000, Rate = 5%, Time = 3 years
 - **Expected Output:** 150.0
23. **Count Substrings in a String:**
- **Question:** Write a function to count the number of occurrences of a substring in a given string.
 - **Logic:** Define a function that uses loops to search for the substring and count occurrences.
 - **Sample Input:** "abababab" and "ab"
 - **Expected Output:** 4
24. **Check for Rotation:**
- **Question:** Write a function to check if one string is a rotation of another string.
 - **Logic:** Define a function that concatenates one string with itself and checks if the other string is a substring.
 - **Sample Input:** "waterbottle" and "erbottlewat"
 - **Expected Output:** Rotation
25. **Find Longest Common Prefix:**
- **Question:** Write a function to find the longest common prefix among an array of strings.
 - **Logic:** Define a function that compares characters of all strings from left to right.
 - **Sample Input:** ["flower", "flour", "flight"]
 - **Expected Output:** "fl"
26. **Compress a String:**
- **Question:** Write a function to compress a string by replacing consecutive characters with their counts.
 - **Logic:** Define a function that uses loops to count consecutive characters and create the compressed string.
 - **Sample Input:** "aaabbbcccc"
 - **Expected Output:** "a3b3c4"

Interview Preparation Codes

27. Reverse Each Word in a String:

- **Question:** Write a function to reverse each word in a string.
- **Logic:** Define a function that splits the string into words, reverses each word, and joins them back together.
- **Sample Input:** "Hello world"
- **Expected Output:** "olleH dlrow"

28. Check for Valid Parentheses:

- **Question:** Write a function to check if a string containing parentheses is valid.
- **Logic:** Define a function that uses a stack to keep track of open and close parentheses.
- **Sample Input:** "{[()]}"
- **Expected Output:** Valid

29. Find the Median of Two Sorted Arrays:

- **Question:** Write a function to find the median of two sorted arrays.
- **Logic:** Define a function that merges the two arrays and calculates the median.
- **Sample Input:** Array 1 = [1, 3], Array 2 = [2]
- **Expected Output:** Median = 2.0

30. Generate a Pattern of Stars:

- **Question:** Write a function to generate a pattern of stars in the form of a right-angled triangle.
- **Logic:** Define a function that uses nested loops to print stars in the desired pattern.
- **Sample Input:** 4
- **Expected Output:**

```
*
**
***
****
```

Interview Preparation Codes

Object-Oriented Programming (OOP), Classes, and Inheritance in Any coding language:

1. Create a Class:

- **Question:** Define a Any coding language class named **Person** with attributes **name** and **age**.
- **Logic:** Use the **class** keyword to create a class and define instance variables for attributes.
- **Sample Input:** N/A
- **Expected Output:** N/A

2. Create an Object:

- **Question:** Create an instance of the **Person** class and assign values to the **name** and **age** attributes.
- **Logic:** Instantiate an object of the class and set attribute values.
- **Sample Input:** `person1 = Person("Alice", 30)`
- **Expected Output:** `person1` is an object with `name="Alice"` and `age=30`.

3. Define Methods:

- **Question:** Add a method **introduce** to the **Person** class that prints a greeting using the **name** attribute.
- **Logic:** Define a method within the class that takes **self** as its first parameter to access instance variables.
- **Sample Input:** `person1.introduce()`
- **Expected Output:** "Hello, my name is Alice."

4. Inheritance:

- **Question:** Create a subclass **Student** that inherits from **Person** and add an additional attribute, **student_id**.
- **Logic:** Define a new class that inherits attributes and methods from the parent class and adds new attributes/methods.
- **Sample Input:** N/A
- **Expected Output:** N/A

5. Method Overriding:

- **Question:** Override the **introduce** method in the **Student** class to include the **student_id** in the greeting.
- **Logic:** Define a method with the same name in the subclass to customize behavior.

Interview Preparation Codes

- **Sample Input:** `student1.introduce()`
- **Expected Output:** "Hello, my name is Alice, and my student ID is 12345."

6. Class Variables:

- **Question:** Add a class variable, `total_students`, to the **Student** class to keep track of the number of student instances.
- **Logic:** Use a class variable to share data among all instances of the class.
- **Sample Input:** `Student.total_students`
- **Expected Output:** Total number of **Student** instances.

7. Static Methods:

- **Question:** Implement a static method, `validate_student_id`, in the **Student** class that checks if a student ID is valid.
- **Logic:** Use the `@staticmethod` decorator to define a static method that doesn't depend on instance-specific data.
- **Sample Input:** `Student.validate_student_id("12345")`
- **Expected Output:** `True` if the student ID is valid, `False` otherwise.

8. Abstract Base Classes:

- **Question:** Create an abstract base class **Shape** with abstract methods `area` and `perimeter` that subclasses like **Circle** and **Rectangle** must implement.
- **Logic:** Use the `abc` module to define abstract base classes with abstract methods.
- **Sample Input:** N/A
- **Expected Output:** N/A

9. Polymorphism:

- **Question:** Implement the `area` and `perimeter` methods in the **Circle** and **Rectangle** subclasses to calculate their respective areas and perimeters.
- **Logic:** Define methods in subclasses with the same names as the abstract methods in the parent class to provide different implementations.
- **Sample Input:** `circle1.area()`, `rectangle1.perimeter()`
- **Expected Output:** Calculated areas and perimeters based on the shape.

10. Composition: - **Question:** Create a **Department** class that contains a list of **Student** objects using composition to represent a "has-a" relationship between classes. - **Logic:** Instantiate objects of one class within another to represent a containment relationship. - **Sample Input:** N/A - **Expected Output:** N/A

Interview Preparation Codes

11. Aggregation: - **Question:** Create a **University** class that contains a list of **Department** objects using aggregation to represent a "has-a" relationship with a weaker connection than composition. - **Logic:** Instantiate objects of one class within another to represent a looser relationship. - **Sample Input:** N/A - **Expected Output:** N/A

12. Operator Overloading: - **Question:** Overload the + operator for a custom class **ComplexNumber** to perform addition of complex numbers. - **Logic:** Define the `__add__` special method in the class to customize the behavior of the + operator. - **Sample Input:** `result = complex1 + complex2` - **Expected Output:** A **ComplexNumber** object representing the sum of two complex numbers.

13. Class Methods: - **Question:** Create a class method `from_string` in a **Date** class that constructs a **Date** object from a string in the format "YYYY-MM-DD". - **Logic:** Use the `@classmethod` decorator to define a class method that constructs objects. - **Sample Input:** `date = Date.from_string("2023-10-25")` - **Expected Output:** A **Date** object representing the specified date.

14. Singleton Pattern: - **Question:** Implement the Singleton design pattern to ensure that only one instance of a class is created. - **Logic:** Use a class variable and a static method to control instance creation and return the existing instance if it exists. - **Sample Input:** N/A - **Expected Output:** N/A

15. Decorators: - **Question:** Create a decorator `@authenticated` that checks if a method can only be accessed by authenticated users. - **Logic:** Define a decorator function that checks user authentication before allowing access to the method. - **Sample Input:** `@authenticated def sensitive_operation():` - **Expected Output:** Controlled access to the method if the user is authenticated.

16. Method Overloading: - **Question:** Implement method overloading for a class to provide different behaviors based on the number of arguments provided. - **Logic:** Use default arguments in the method to handle different argument combinations. - **Sample Input:** Varies based on overloading. - **Expected Output:** Different behaviors based on the number and type of arguments provided.

17. Multiple Inheritance: - **Question:** Create a class **Employee** and a class **Manager**, and have a class **ManagerEmployee** inherit from both. - **Logic:** Implement multiple inheritance to inherit attributes and methods from multiple classes. - **Sample Input:** N/A - **Expected Output:** N/A

18. Factory Method Pattern: - **Question:** Implement the Factory Method design pattern to create objects of different types without specifying their exact class. - **Logic:** Define a factory method in a class to create objects of a related class. - **Sample Input:** N/A - **Expected Output:** N/A

Interview Preparation Codes

19. Dependency Injection: - **Question:** Implement dependency injection by passing dependencies (objects or functions) as parameters to a class. - **Logic:** Define a class that accepts dependencies as parameters and uses them to perform its tasks. - **Sample Input:** Varies based on dependencies provided. - **Expected Output:** The class uses the provided dependencies to perform its operations.

20. Observer Pattern: - **Question:** Implement the Observer design pattern to allow an object (subject) to notify its dependents (observers) of changes. - **Logic:** Define a subject class that maintains a list of observers and notifies them of state changes. - **Sample Input:** N/A - **Expected Output:** N/A

21. Template Method Pattern: - **Question:** Implement the Template Method design pattern to define the skeleton of an algorithm in a method and allow subclasses to provide their own implementations for specific steps. - **Logic:** Define a template method with the algorithm's structure and abstract methods that subclasses must implement. - **Sample Input:** N/A - **Expected Output:** N/A

22. Strategy Pattern: - **Question:** Implement the Strategy design pattern to define a family of algorithms, encapsulate each one, and make them interchangeable. - **Logic:** Define a context class that can switch between different strategy objects. - **Sample Input:** N/A - **Expected Output:** N/A

23. Command Pattern: - **Question:** Implement the Command design pattern to encapsulate a request as an object, thereby allowing for parameterization of clients with queuing, requests, and operations. - **Logic:** Define a command interface and concrete command classes that encapsulate actions. - **Sample Input:** N/A - **Expected Output:** N/A

24. State Pattern: - **Question:** Implement the State design pattern to allow an object to alter its behavior when its internal state changes. - **Logic:** Define state classes and a context class that can switch between states. - **Sample Input:** N/A - **Expected Output:** N/A

25. Builder Pattern: - **Question:** Implement the Builder design pattern to construct a complex object step by step. - **Logic:** Define a builder interface and concrete builder classes for different variations of an object. - **Sample Input:** N/A - **Expected Output:** N/A

26. Chain of Responsibility: - **Question:** Implement the Chain of Responsibility design pattern to pass a request along a chain of handlers. - **Logic:** Define a chain of handler objects where each handler decides whether to process the request or pass it to the next handler. - **Sample Input:** N/A - **Expected Output:** N/A

Interview Preparation Codes

27. Composite Pattern: - **Question:** Implement the Composite design pattern to compose objects into tree structures to represent part-whole hierarchies. - **Logic:** Define component and leaf classes, where leaf objects can be composed into composite objects. - **Sample Input:** N/A - **Expected Output:** N/A

28. Prototype Pattern: - **Question:** Implement the Prototype design pattern to create new objects by copying an existing object. - **Logic:** Define a prototype interface and concrete prototype classes. - **Sample Input:** N/A - **Expected Output:** N/A

29. Visitor Pattern: - **Question:** Implement the Visitor design pattern to represent an operation to be performed on elements of an object structure. - **Logic:** Define visitor and element classes where visitors can visit elements and perform operations. - **Sample Input:** N/A - **Expected Output:** N/A

30. Memento Pattern: - **Question:** Implement the Memento design pattern to capture and externalize an object's internal state so the object can be restored to that state later. - **Logic:** Define a memento class to capture and restore the state of an object. - **Sample Input:** N/A - **Expected Output:** N/A

Interview Preparation Codes

Stack Data Structure:

1. Implement a Stack:

- Create a Any coding language class to implement a stack data structure with methods like **push**, **pop**, **peek**, and **is_empty**.
- Use a list to simulate a stack and implement the methods accordingly.

2. Check Balanced Parentheses:

- Write a Any coding language function to check if a given string containing parentheses is balanced (i.e., every opening parenthesis has a closing one).
- Use a stack to keep track of open parentheses and ensure they are closed properly.

3. Evaluate Postfix Expression:

- Write a Any coding language function to evaluate a postfix expression (e.g., "23*5+") using a stack.
- Use a stack to process the postfix expression and calculate the result.

4. Implement a Min Stack:

- Create a Any coding language stack that supports the **push**, **pop**, **top**, and **get_min** operations in $O(1)$ time.
- Use an additional stack to keep track of the minimum element at any given time.

5. Reverse a Stack:

- Write a Any coding language function to reverse the order of elements in a stack.
- Use an auxiliary stack to reverse the elements from the original stack.

6. Implement Two Stacks in One Array:

- Create a Any coding language class that implements two stacks in a single array efficiently.
- Divide the array into two halves and manage two stacks independently.

7. Sort a Stack:

- Write a Any coding language function to sort a stack using only one additional stack.
- Use an auxiliary stack to perform the sorting operation.

8. Implement a Max Stack:

- Create a Any coding language stack that supports the **push**, **pop**, **top**, and **get_max** operations in $O(1)$ time.
- Use an additional stack to keep track of the maximum element at any given time.

Queue Data Structure:

9. Implement a Queue:

Interview Preparation Codes

- Create a Any coding language class to implement a queue data structure with methods like **enqueue**, **dequeue**, **peek**, and **is_empty**.
 - Use a list to simulate a queue and implement the methods accordingly.
10. **Implement a Circular Queue:**
- Create a Any coding language class to implement a circular queue data structure with methods like **enqueue**, **dequeue**, and checks for full and empty states.
 - Use an array and maintain front and rear pointers.
11. **Implement a Queue using Stacks:**
- Create a Any coding language queue using two stacks and implement the **enqueue** and **dequeue** operations.
 - Use two stacks to simulate the queue and perform enqueue and dequeue operations.
12. **Implement a Double-Ended Queue (Deque):**
- Create a Any coding language class to implement a deque data structure with methods like **append**, **appendleft**, **pop**, and **popleft**.
 - Use a doubly-linked list to implement the deque.
13. **Implement a Priority Queue:**
- Create a Any coding language priority queue data structure that supports **insert** and **get_highest_priority** operations.
 - Use a heap data structure to implement the priority queue.
14. **Reorder a Queue:**
- Write a Any coding language function to reorder a given queue such that elements with even values appear before elements with odd values, without using any extra data structures.
 - Use two queues to separate even and odd elements.
15. **Implement a Blocking Queue:**
- Create a Any coding language blocking queue that supports **put** and **get** operations, which block when the queue is empty or full.
 - Use threading or multiprocessing to implement the blocking behavior.
16. **Implement a Queue using Two Stacks:**
- Create a Any coding language queue using two stacks and implement the **enqueue** and **dequeue** operations.
 - Use two stacks to simulate the queue and perform enqueue and dequeue operations.
17. **Generate Binary Numbers using a Queue:**
- Write a Any coding language function to generate binary numbers from 1 to n using a queue.
 - Use a queue to implement a breadth-first search (BFS) approach to generate binary numbers.
18. **Implement a Sliding Window Maximum:**
- Write a Any coding language function to find the maximum element in a sliding window of fixed size in an array.

Interview Preparation Codes

- Use a deque to maintain the maximum values within the sliding window.

Linked List Basics:

1. Implement a Singly Linked List:

- Create a Any coding language class to represent a singly linked list with methods like **insert_at_head**, **insert_at_tail**, **delete**, and **search**.
- Implement the methods to manipulate the linked list accordingly.

2. Find the Length of a Linked List:

- Write a function to find the length of a singly linked list.
- Traverse the linked list and count the number of nodes.

3. Reverse a Linked List:

- Write a function to reverse a singly linked list.
- Use iterative or recursive methods to reverse the pointers of the nodes.

4. Detect a Cycle in a Linked List:

- Write a function to detect if a singly linked list has a cycle.
- Use Floyd's cycle-finding algorithm (two-pointer technique) to detect cycles in the linked list.

5. Find the Middle of a Linked List:

- Write a function to find the middle node of a singly linked list.
- Use the slow and fast pointer approach to find the middle node.

Advanced Linked List Problems:

6. Remove Nth Node from End of List:

- Write a function to remove the nth node from the end of a singly linked list.
- Use two pointers to find the appropriate node and remove it.

7. Merge Two Sorted Linked Lists:

- Write a function to merge two sorted singly linked lists into a single sorted list.
- Compare nodes of both lists and merge them into a new linked list.

8. Intersection Point in Y Shaped Linked Lists:

- Write a function to find the node where two linked lists intersect.
- Traverse both lists, find their lengths, align them, and then find the intersection node.

9. Detect a Cycle's Starting Point:

- Write a function to find the starting node of a cycle in a linked list.
- Use Floyd's cycle-finding algorithm and then move one pointer to the start and iterate until they meet.

10. Palindrome Linked List:

Interview Preparation Codes

- Write a function to check if a linked list is a palindrome.
- Reverse the second half of the linked list and compare it with the first half.

11. Flatten a Multilevel Doubly Linked List:

- Write a function to flatten a multilevel doubly linked list into a singly linked list.
- Flatten the list while maintaining the next and child pointers.

Linked List Operations:

12. Add Two Numbers Represented by Linked Lists:

- Write a function to add two numbers represented by linked lists.
- Traverse both lists simultaneously, calculate the sum, and create a new linked list for the result.

13. Intersection of Two Linked Lists:

- Write a function to find the node where two linked lists intersect.
- Find the lengths of both lists, align them, and then traverse until the intersection point is found.

14. Remove Duplicates from Sorted List:

- Write a function to remove duplicates from a sorted linked list.
- Traverse the list and remove duplicates by adjusting the pointers.

15. Rotate a Linked List:

- Write a function to rotate a linked list by k places.
- Find the length, calculate the new head, and adjust pointers accordingly.

Special Types of Linked Lists:

16. Clone a Linked List with Next and Random Pointer:

- Write a function to clone a linked list with next and random pointers.
- Create new nodes and map the relationships between original and new nodes.

17. LRU Cache Implementation using Linked List:

- Implement an LRU (Least Recently Used) cache using a combination of a linked list and a dictionary.
- Maintain the order of items based on usage and update the cache accordingly.

18. Flatten a Binary Tree to Linked List:

- Write a function to flatten a binary tree into a singly linked list in-place.
- Use a recursive approach to flatten the binary tree.

Variations of Linked Lists:

19. Add Two Numbers Represented by Reverse Linked Lists:

Interview Preparation Codes

- Write a function to add two numbers represented by reverse linked lists.
- Traverse both lists simultaneously, calculate the sum, and create a new linked list for the result.

20. Rearrange a Linked List in Zigzag Fashion:

- Write a function to rearrange a linked list in a zigzag fashion.
- Traverse the list and reorder nodes based on the given pattern.

21. Rotate a Linked List:

- Write a function to rotate a linked list by k nodes from the end.
- Use the length of the list to find the new head after rotation.

Miscellaneous Linked List Problems:

22. Partition a Linked List around a Given Value:

- Write a function to partition a linked list around a given value x.
- Maintain two separate linked lists for nodes smaller and greater than x, then concatenate them.

23. Convert Binary Number in a Linked List to Integer:

- Write a function to convert a binary number represented by a linked list to an integer.
- Traverse the list, convert the binary number, and return the integer value.

24. Add One to a Linked List Representing a Number:

- Write a function to add one to a number represented by a linked list.
- Reverse the list, add one, and handle carry appropriately.

25. Copy Random Pointer Linked List:

- Write a function to copy a linked list with next and random pointers in $O(n)$ time.
- Create a new node for each original node and use a dictionary to map original nodes to new nodes.

Interview Preparation Codes

Binary Tree Basics:

1. **Implement a Binary Tree:**
 - Create a Any coding language class to represent a binary tree with methods to insert and traverse nodes.
 - Implement the methods to manipulate and traverse the binary tree.
2. **Find the Height of a Binary Tree:**
 - Write a function to find the height (depth) of a binary tree.
 - Use a recursive approach to traverse the tree and calculate the height.
3. **Check if a Binary Tree is Balanced:**
 - Write a function to check if a binary tree is balanced (the heights of the two subtrees of every node never differ by more than one).
 - Use a recursive approach to check the balance of each node.
4. **Serialize and Deserialize a Binary Tree:**
 - Implement functions to serialize and deserialize a binary tree.
 - Use a depth-first search (DFS) or breadth-first search (BFS) approach to serialize and deserialize.
5. **Find the Lowest Common Ancestor in a Binary Tree:**
 - Write a function to find the lowest common ancestor of two nodes in a binary tree.
 - Use a recursive approach to search for the nodes and their common ancestor.

Binary Search Tree (BST):

6. **Implement a Binary Search Tree (BST):**
 - Create a Any coding language class to represent a binary search tree with methods to insert, search, and delete nodes.
 - Implement the methods to manipulate and search in the BST.
7. **Validate a Binary Search Tree (BST):**
 - Write a function to determine if a binary tree is a valid binary search tree.
 - Use a recursive approach to validate the properties of a BST.
8. **Find the Inorder Successor in a BST:**
 - Write a function to find the inorder successor of a given node in a BST.
 - Use a recursive approach to find the successor.
9. **Convert a Sorted Array to a Balanced BST:**
 - Write a function to convert a sorted array into a balanced binary search tree.

Interview Preparation Codes

- Use a recursive approach to build the tree from the sorted array.

10. Recover a BST with Two Swapped Nodes:

- Write a function to recover a binary search tree where two nodes are swapped.
- Use an inorder traversal to detect the swapped nodes and correct them.

Tree Traversals:

11. Preorder Traversal:

- Write a function to perform a preorder traversal of a binary tree.
- Use a recursive approach to visit nodes in the order: root, left, right.

12. Inorder Traversal:

- Write a function to perform an inorder traversal of a binary tree.
- Use a recursive approach to visit nodes in the order: left, root, right.

13. Postorder Traversal:

- Write a function to perform a postorder traversal of a binary tree.
- Use a recursive approach to visit nodes in the order: left, right, root.

14. Level-Order Traversal:

- Write a function to perform a level-order traversal of a binary tree.
- Use a queue to visit nodes level by level.

Advanced Tree Problems:

15. Find the Diameter of a Binary Tree:

- Write a function to find the diameter (longest path) of a binary tree.
- Use a recursive approach to calculate the diameter.

16. Count the Number of Unival Subtrees:

- Write a function to count the number of unival (same value) subtrees in a binary tree.
- Use a recursive approach to check for unival subtrees.

17. Check if a Binary Tree is Symmetric:

- Write a function to check if a binary tree is symmetric (a mirror image of itself).
- Use a recursive approach to check for symmetry.

18. Flatten a Binary Tree into a Linked List:

- Write a function to flatten a binary tree into a singly linked list in-place.
- Use a recursive approach to flatten the binary tree.

19. Find All Paths in a Binary Tree:

- Write a function to find all paths from the root to leaf nodes in a binary tree.
- Use a recursive approach to collect paths.

Interview Preparation Codes

Special Types of Trees:

20. Convert Sorted Linked List to Binary Search Tree:

- Write a function to convert a sorted linked list into a balanced binary search tree.
- Use a recursive approach to build the tree from the linked list.

21. Check if a Binary Tree is Subtree of Another Binary Tree:

- Write a function to check if one binary tree is a subtree of another binary tree.
- Use a recursive approach to search for the subtree.

22. Serialize and Deserialize N-ary Tree:

- Implement functions to serialize and deserialize an N-ary tree.
- Use a depth-first search (DFS) approach for serialization and deserialization.

Variations of Tree Problems:

23. Find the kth Smallest Element in a BST:

- Write a function to find the kth smallest element in a binary search tree.
- Use an inorder traversal with a counter to find the kth smallest element.

24. Convert Binary Tree to Doubly Linked List:

- Write a function to convert a binary tree into a doubly linked list in-place.
- Use a recursive approach to convert the tree.

25. Count the Number of Nodes in a Complete Binary Tree:

- Write a function to count the number of nodes in a complete binary tree.
- Use a binary search approach to count nodes efficiently.

Miscellaneous Tree Problems:

26. Implement an Expression Tree:

- Create a Any coding language class to represent an expression tree and evaluate expressions.
- Use a stack to build and evaluate expressions.

27. Find the Closest Leaf in a Binary Tree:

- Write a function to find the closest leaf node to a given target node in a binary tree.
- Use a depth-first search to find the closest leaf.

28. Sum of All Left Leaves in a Binary Tree:

- Write a function to find the sum of all left leaves in a binary tree.
- Use a depth-first search to identify left leaves and calculate the sum.

29. Construct Binary Tree from Preorder and Inorder Traversal:

Interview Preparation Codes

- Write a function to construct a binary tree from its preorder and inorder traversals.
- Use recursive methods to construct the tree.

30. **Path Sum III:**

- Write a function to count the total number of paths that sum to a given value in a binary tree.
- Use a recursive approach to explore paths and count valid sums.

Interview Preparation Codes

Graph Basics:

1. **Implement an Undirected Graph:**
 - Create a Any coding language class to represent an undirected graph with methods to add vertices and edges.
 - Implement the methods to manipulate the graph.
2. **Implement a Directed Graph:**
 - Create a Any coding language class to represent a directed graph with methods to add vertices and directed edges.
 - Implement the methods to manipulate the directed graph.
3. **Depth-First Search (DFS):**
 - Write a function to perform a depth-first search on a graph.
 - Use recursion to traverse the graph and visit nodes in a specific order.
4. **Breadth-First Search (BFS):**
 - Write a function to perform a breadth-first search on a graph.
 - Use a queue to traverse the graph level by level.
5. **Topological Sort:**
 - Write a function to perform a topological sort on a directed acyclic graph (DAG).
 - Use DFS or BFS to obtain a topological order of the nodes.

Shortest Paths:

6. **Dijkstra's Algorithm:**
 - Implement Dijkstra's algorithm to find the shortest path in a weighted graph with non-negative edge weights.
 - Use a priority queue or min-heap to find the shortest path.
7. **Bellman-Ford Algorithm:**
 - Implement the Bellman-Ford algorithm to find the shortest path in a weighted graph with negative edge weights and detect negative cycles.
 - Use dynamic programming and relaxation steps.
8. **Floyd-Warshall Algorithm:**
 - Implement the Floyd-Warshall algorithm to find all shortest paths in a graph.
 - Use dynamic programming to calculate the shortest paths between all pairs of nodes.
9. **Single-Source Shortest Path in a DAG:**
 - Write a function to find the shortest paths from a single source node in a directed acyclic graph (DAG).
 - Use topological sorting and dynamic programming.

Graph Connectivity:

10. **Connected Components:**

Interview Preparation Codes

- Write a function to find and count the connected components in an undirected graph.
- Use DFS or BFS to explore the graph.
- 11. Minimum Spanning Tree (Prim's Algorithm):**
 - Implement Prim's algorithm to find the minimum spanning tree of an undirected, weighted graph.
 - Use a priority queue or min-heap to grow the minimum spanning tree.
- 12. Minimum Spanning Tree (Kruskal's Algorithm):**
 - Implement Kruskal's algorithm to find the minimum spanning tree of an undirected, weighted graph.
 - Use a disjoint-set (union-find) data structure to grow the minimum spanning tree.
- 13. Strongly Connected Components (Kosaraju's Algorithm):**
 - Write a function to find and count strongly connected components in a directed graph.
 - Use two DFS traversals to determine strongly connected components.

Advanced Graph Problems:

- 14. Word Ladder:**
 - Write a function to find the length of the shortest transformation sequence from the start word to the end word by changing one character at a time.
 - Use BFS to explore word transformations.
- 15. Minimum Cut in a Graph:**
 - Implement the Karger's algorithm to find the minimum cut in an undirected graph.
 - Use random contractions to identify the minimum cut.
- 16. Max Flow in a Network (Ford-Fulkerson Algorithm):**
 - Implement the Ford-Fulkerson algorithm to find the maximum flow in a network.
 - Use augmenting paths to increase the flow.
- 17. Bipartite Graph Check:**
 - Write a function to check if a graph is bipartite (can be divided into two sets such that there are no edges within a set).
 - Use BFS or DFS to assign colors to nodes and check for conflicts.

Graph Applications:

- 18. Shortest Path in a Grid:**
 - Write a function to find the shortest path in a grid with obstacles.
 - Use BFS or Dijkstra's algorithm to navigate the grid.
- 19. Minimum Height Trees:**

Interview Preparation Codes

- Write a function to find the minimum height trees in a tree-like undirected graph.
- Use a topological sorting approach.

20. Redundant Connection in a Directed Graph:

- Write a function to find a redundant connection in a directed graph.
- Use DFS or union-find to detect cycles and find the redundant edge.

Variations of Graph Problems:

21. Largest Component Size by Common Factor:

- Write a function to find the size of the largest component in an undirected graph where two numbers are connected if they share a common factor.
- Use graph traversal to find the connected components.

22. Course Schedule (Directed Graph):

- Write a function to check if it's possible to finish all courses in a course schedule based on prerequisites.
- Use a directed graph and topological sorting.

23. Find the Celebrity:

- Write a function to find the celebrity in a party, where everyone knows the celebrity, but the celebrity doesn't know anyone.
- Use a graph-based elimination approach.

Miscellaneous Graph Problems:

24. Valid Tree:

- Write a function to check if a given graph is a valid tree.
- Use graph properties to check for cycles and connectivity.

25. Detect Cycles in a Directed Graph:

- Write a function to detect cycles in a directed graph.
- Use DFS to identify back edges in the graph.

26. Maximal Network Rank:

- Write a function to find the maximal network rank in a graph.
- Use degree calculations and adjacency lists.

27. Minimum Knight Moves:

- Write a function to find the minimum number of moves required for a knight to move from one cell to another on a chessboard.
- Use BFS to explore possible moves.

28. Number of Operations to Make Network Connected:

- Write a function to find the minimum number of operations needed to make a network connected.
- Use disjoint-set (union-find) data structure to determine connected components.

29. Redundant Connection (Undirected Graph):

Interview Preparation Codes

- Write a function to find a redundant connection in an undirected graph.
- Use union-find to detect cycles and find the redundant edge.

30. Evaluate Division:

- Write a function to evaluate division equations given as pairs of strings and their values.
- Use a graph-based approach to perform division operations.